Lab2：BombLab 实验报告

计算机系统基础（2026 秋）

1 Phase 1：函数调用

1.1 关键汇编与分析

在 `phase_01` 中：

```
mov $0x1,%esi
call valid_u32_fields
...
mov %r12,%rdi
call p1_compute
cmp %eax,-0x14(%rbp)
sete %al
```

由 `valid_u32_fields` 的参数可知，本关需要一个无符号整数。输入与 `p1_compute` 的返回值比较，因此需要恢复该函数。

`p1_compute` 依次调用三个辅助函数。

其中：

```
# p1_step1
lea (%rdi,%rsi,2),%eax
xor $0xa10f,%eax

# p1_step0
lea (%rdi,%rdi,2),%eax
add $0xa91d,%esi
xor %esi,%eax

# p1_step2
sub %esi,%edi
lea 0x6142(%rdi,%rdi,4),%eax
```

`lea` 在这里用于整数运算。例如 `(%rdi,%rdi,2)` 相当于 3 rdi。

1.2 恢复的计算关系

设

$$h = (uint32_t)(c \gg 32), \qquad l = (uint32_t)c.$$

三个辅助函数对应：

$$a = (h + 2(c \gg 16)) \oplus 0xa10f,$$
$$b = (3l) \oplus (h + 0xa91d),$$
$$p_0 = 5(b - a) + 0x6142.$$

因此最终输出一个整数：

$$\boxed{values[0] = p_0}$$

所有中间计算均保留低 32 位。

2 Phase 2：数组与递推

2.1 关键汇编

```
mov $0x6,%esi
call valid_u32_fields
...
shr $0x9,%rax
and $0x7f,%eax
lea (%rax,%rax,2),%eax
```

可知本关需要六个整数，且第一个整数为：

$$\boxed{p_0 = 3((c \gg 9)\ \&\ 0x7f)}$$

随后程序进入循环：

```
lea 0xc(%rdx),%ecx
mov %r13,%rax
shr %cl,%rax
lea 0xf(%rdx),%ecx
and $0x1f,%eax
xor %ecx,%eax
lea (%rsi,%rsi,4),%ecx
mov -0x40(%rbp,%rdx,4),%esi
add %ecx,%eax
cmp %eax,%esi
```

2.2 计算关系

循环中 `rdx` 充当下标 i，`rsi` 保存前一项。

`lea (%rsi,%rsi,4)` 表示前一项乘以 5。

因此：

$$\boxed{p_i = 5p_{i-1} + [((c \gg (i + 12))\ \&\ 31) \oplus (i + 15)]}$$

其中 $1 \le i \le 5$。

对应的 C 语言实现为：

```
values[0] = 3 * ((c >> 9) & 0x7f);

for (uint64_t i = 1; i < 6; i++) {
    values[i] = (uint32_t)(
        5 * values[i - 1] +
        (((c >> (i + 12)) & 0x1f) ^ (i + 15))
    );
}
```

3 Phase 3：跳转表

3.1 输入与分支选择

首先观察到：

```
mov %r12d,%edi
and $0x7,%edi
cmp %edi,-0x18(%rbp)
```

因此第一个整数为

$$\boxed{p_0 = c \,\&\, 7}$$

另一个参数由以下汇编计算：

```
mov %r12,%rax
mov %r12,%rsi
shr $0x3,%rax
shr $0x20,%rsi
xor %eax,%esi
call p3_dispatch
```

即

$$k = (uint32_t)(c \gg 32) \oplus (uint32_t)(c \gg 3).$$

3.2 恢复跳转表

在 `p3_dispatch` 中：

```
cmp $0x7,%eax
ja  0x402225
notrack jmp *0x4057e0(,%rax,8)
```

这里不是普通条件分支，而是根据下标从内存读取跳转地址。

通过 GDB：

```
(gdb) x/8ga 0x4057e0
```

读取后发现，实际分支顺序并不等于函数命名顺序。

下标 p_0	实际调用函数
0	p3_case_2
1	p3_case_1
2	p3_case_6
3	p3_case_5
4	p3_case_7
5	p3_case_4
6	p3_case_3
7	p3_case_0

表 1: Phase 3 跳转表

3.3 计算关系

分别恢复八个 case 函数：

$$p_1 = \begin{cases} (2k) \oplus 0x83c7, & p_0 = 0, \\ (k \oplus 0x1afa) + (k \gg 2), & p_0 = 1, \\ 5k + 0x1766f, & p_0 = 2, \\ (k \oplus 0x5b22) + (k \gg 2), & p_0 = 3, \\ 3k + 0xeee7, & p_0 = 4, \\ (2k) \oplus 0x8d21, & p_0 = 5, \\ 5k + 0x2c51, & p_0 = 6, \\ 5k + 0xcbd4, & p_0 = 7. \end{cases}$$

最终：

$$\boxed{values[0] = p_0, \quad values[1] = p_1}$$

在 C 语言中使用 `switch` 实现这些分支。

4 Phase 4：递归函数

4.1 参数恢复

```
mov %r12,%rcx
mov %r12,%rax
mov %r12d,%edi
xor %esi,%esi
```

```
shr $0x20,%rax
shr $0x7,%rcx
and $0x7f,%edi
mov $0x7f,%edx
xor %eax,%ecx
call p4_walk
```

根据调用约定，得到：

$$\begin{aligned} t &= c \,\&\, 127, \\ low = 0, &\qquad high = 127, \\ k &= (uint32_t)(c \gg 7) \oplus (uint32_t)(c \gg 32). \end{aligned}$$

4.2 递归关系

分析 `p4_walk`：

```
mov %edx,%edi
sub %esi,%edi
shr $1,%edi
add %esi,%edi
cmp %eax,%edi
je  0x402310
```

对应二分查找的中点：

$$m = low + \left\lfloor \frac{high - low}{2} \right\rfloor.$$

若 $t = m$，执行：

```
lea 0x90(%rdi),%eax
xor %ecx,%eax
ret
```

得到

$$F(t, low, high, k) = (m + 0x90) \oplus k.$$

左、右分支分别递归访问 $[low, m-1]$ 和 $[m+1, high]$。

递归返回后执行：

```
lea (%rax,%rax,2),%eax
xor %ebx,%eax
xor $0x17,%eax   # left
```

右分支的最后常数为 `0x1d`。

于是恢复：

$$F(t,l,h,k)=\begin{cases}(m+0x90)\oplus k, & t=m,\\ 3F(t,l,m-1,k)\oplus(m+k)\oplus 0x17, & t<m,\\ 3F(t,m+1,h,k)\oplus(m+k)\oplus 0x1d, & t>m.\end{cases}$$

本关在 `defuser.c` 中使用独立递归函数实现。

它不仅进行二分查找，还会在递归返回过程中利用中点与分支信息不断修改返回值。

5 Phase 5：结构体与链表

5.1 输出关系

```
lea -0x14(%rbp),%rsi
mov %r12,%rdi
call p5_traverse
mov %eax,%edx
cmp %edx,-0x1c(%rbp)
...
mov -0x14(%rbp),%eax
cmp %eax,-0x18(%rbp)
```

说明 `p5_traverse` 返回第一个口令，并通过指针参数写出第二个口令。

5.2 恢复结构体

核心指令：

```
mov (%rdx),%rdx
mov 0x8(%rdx),%esi
add 0xc(%rdx),%eax
```

其中 `rdx` 保存节点指针。

可以推断结构体布局：

偏移	大小	含义
0	8 字节	next 指针
8	4 字节	value
12	4 字节	extra

表 2: Phase 5 链表节点布局

5.3 恢复静态数据

程序通过：

```
and $0x7,%eax
```

```
mov 0x405b20(,%rax,8),%rdx
```

根据 challenge 的低三位选取八个起始节点之一。

使用：

```
(gdb) x/8gx 0x405b20
(gdb) x/48wx 0x405b60
```

分别读取起始指针和十二个节点。

为避免在 defuser.c 中直接使用原程序的内存地址，将十二个节点映射为数组下标。

```
const uint32_t heads[8] = {
    6, 2, 4, 1, 10, 11, 7, 5
};
```

其中节点数组下标 0 对应原程序地址 0x405b60，每增加一个下标，原始地址增加 16 字节。

5.4 计算关系

```
shr $0x20,%rdi
shr $0x6,%rax
xor %edi,%eax
xor $0x6c59,%eax
lea (%rax,%rax,2),%eax
add 0xc(%rdx),%eax
xor %esi,%eax
```

令

$$
\begin{aligned}
i &= c \,\&\, 7, \\
s &= (c \gg 3) \,\&\, 3, \\
k &= (uint32_t)(c \gg 6) \oplus (uint32_t)(c \gg 32) \oplus 0x6c59.
\end{aligned}
$$

从 heads[i] 对应的节点开始：

$$
acc = (3k + extra) \oplus value.
$$

随后沿 next 移动 s 次，每次执行：

$$
\boxed{acc \leftarrow (3acc + extra) \oplus value}
$$

最后：

$$
\boxed{values[0] = acc, \qquad values[1] = \text{最后访问节点的 value}}
$$

5.5 调试说明

本关调试中曾遇到静态数据恢复错误。

例如节点 0x405b80 的 extra 为 0x0780，不能误写为 0x7800。

此外，部分链表路径会访问最初没有读取到的节点。

重新检查完整的十二个节点之后，使用数组下标模拟原始链表，最终通过公开测试。

6 Phase 6：函数指针表

6.1 两次间接调用

在 p6_dispatch 中：

```
shr $0x6,%rdi
shr $0x20,%rax
xor %eax,%edi
...
and $0x38,%ebx
and $0x7,%eax
call *0x405dc0(,%rax,8)
xor $0xa712,%eax
mov %eax,%edi
call *0x405dc0(%rbx)
add $0xeac0,%eax
```

首先计算：

$$x = (uint32_t)(c \gg 6) \oplus (uint32_t)(c \gg 32).$$

两个函数的下标分别由 challenge 的低六位决定：

$$i = c \,\&\, 7, \qquad j = (c \gg 3) \,\&\, 7.$$

6.2 恢复函数指针表

通过：

```
(gdb) x/8ga 0x405dc0
```

得到：

下标	0	1	2	3	4	5	6	7
函数编号	1	2	2	4	0	3	5	1

表 3: Phase 6 函数指针表

表中共有八项，但只有六种运算。

逐个反汇编 p6_op_0 到 p6_op_5，恢复各函数的计算规则。

其中包含两种循环移位：

$$ROR_{32}(x,3) = (x \gg 3) \mid (x \ll 29),$$

$$ROL_{32}(x,7) = (x \ll 7) \mid (x \gg 25).$$

完整的按下标计算函数为：

$$f_i(x) = \begin{cases} ROR_{32}(x,3), & i = 0,7, \\ ((x \ll 7) + 0x8fb0) \oplus 7x, & i = 1,2, \\ ROL_{32}(x,7) \oplus 0xae66, & i = 3, \\ (x \oplus 0x2608) + 2x, & i = 4, \\ (x + 0x5f05) \oplus (x \gg 5), & i = 5, \\ ((x \gg 5) \oplus x) + 0x9919, & i = 6. \end{cases}$$

6.3 最终计算关系

第一次间接调用得到 $f_i(x)$，将返回值异或 `0xa712`，再作为第二个函数的输入。

最终口令为：

$$\boxed{p_0 = f_j\left(f_i(x) \oplus 0xa712\right) + 0xeac0}$$

在 C 语言中使用辅助函数和 `switch` 实现等价逻辑。

7 Secret：隐藏关

7.1 字符串格式

隐藏关包含 `secret_gate`、`secret_copy`、`secret_phase` 和 `secret_mix`。

在 `secret_gate` 中：

```
call strlen@plt
sub $0x24,%rax
cmp $0x7f,%rax
jbe 0x402700
```

由于使用无符号比较，可得字符串长度要求：

$$36 \le strlen(input) \le 163.$$

随后：

```
cmpl $0x4e45504f,-0xb0(%rbp)
jne ...
```

复制缓冲区起始位置为 `rbp-0xd0`，检查位置为 `rbp-0xb0`，偏移恰好为 32 字节。

结合小端存储，`0x4e45504f` 对应字符串 `OPEN`。

因此输入结构为：

```
[32 bytes]OPEN[p0] [p1] [p2]
```

其中三个整数从偏移 36 处开始解析。

7.2 恢复计算关系

设

$$
\begin{aligned}
c &= challenge, \qquad t = context,\\
a &= (uint32_t)(c \gg 32) \oplus (uint32_t)c,\\
b &= (uint32_t)(t \gg 32) \oplus (uint32_t)t.
\end{aligned}
$$

第一项：

$$\boxed{p_0 = t \,\&\, 3}$$

`secret_mix` 的汇编为：

```
mov %edi,%eax
ror $0x5,%eax
xor $0xe1b2,%eax
ret
```

因此定义：

$$M(x) = ROR_{32}(x, 5) \oplus 0xe1b2.$$

再令：

$$d = (0x65e8p_0) \oplus (uint32_t)(t \gg 16).$$

第二、三项分别为：

$$\boxed{p_1 = M(a \oplus b) \oplus d}$$

$$\boxed{p_2 = p_1 \oplus (b + p_0 + 0x845a + (a \gg 5))}$$

最后使用 `snprintf` 输出包含固定前缀与三个无符号整数的字符串。

8 测试与验证

8.1 公开测试结果

执行：

```
make
python3 public_test.py
```

得到：

Public tests: 60/60

以及：

Secret public tests: 10/10

所有公开测试均通过。

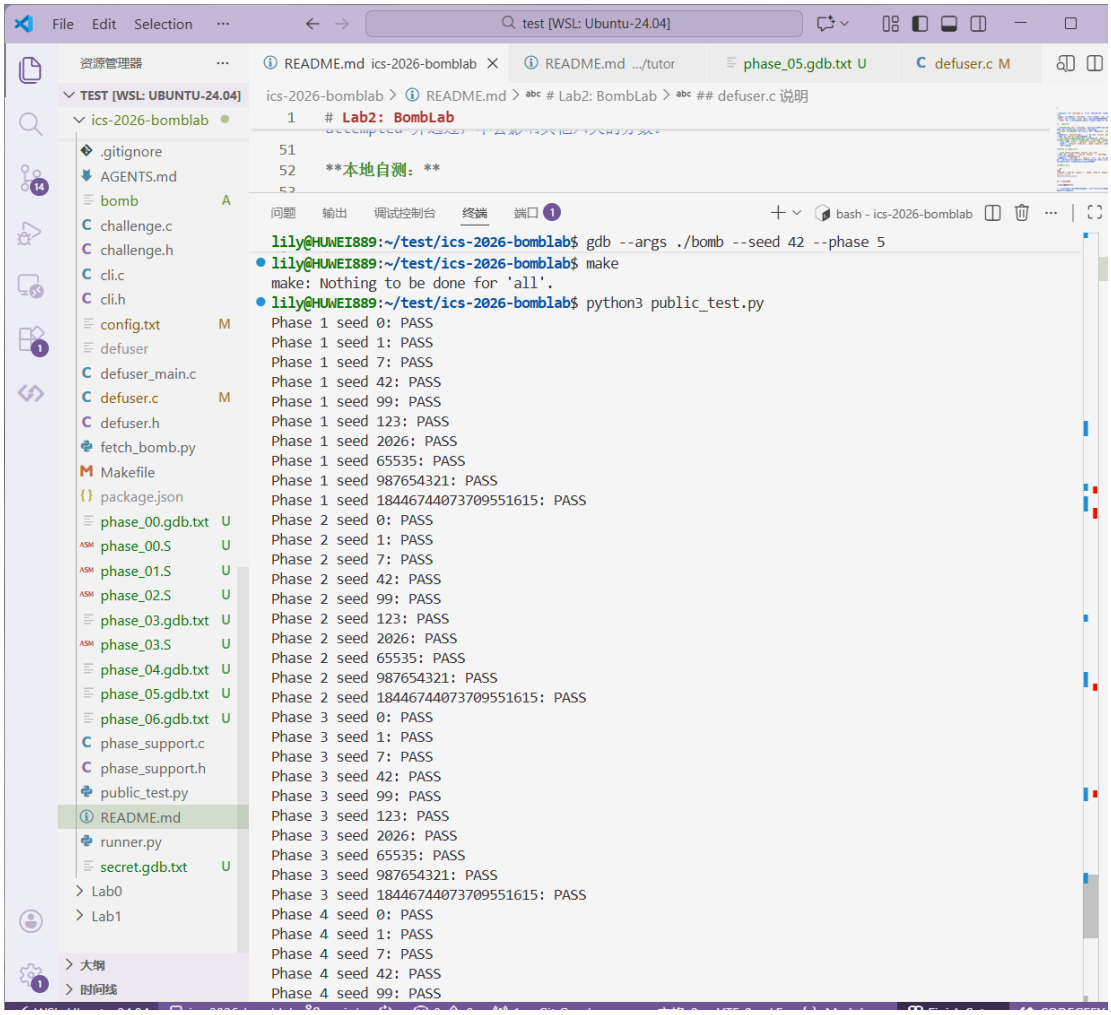


图 1: 公开测试记录（一）

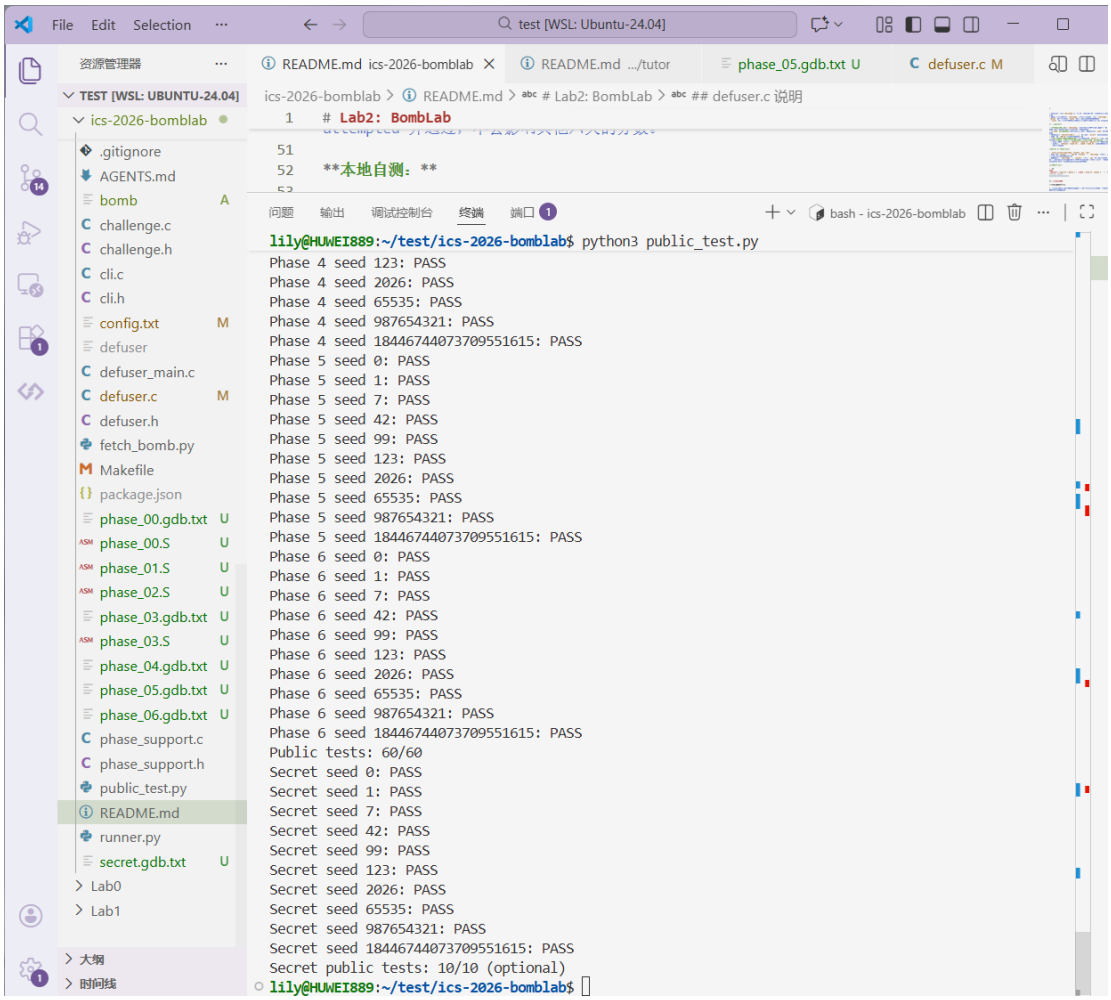


图 2: 公开测试记录（二）：总结果与 Secret

8.2 非公开 seed 验证

除了公开列表中的 seed，还使用了：

$$\boxed{seed = 114514}$$

执行：

```
./defuser --seed 114514
```

得到六个常规关卡的答案：

```
2186505535
237 1189 5956 29811 149060 745315
5 974204681
703942541
1385719713 296
1109584457
```

之后使用：

```
./bomb --seed 114514
```

逐关输入并验证。

结果如下：

关卡	seed	结果
Phase 1	114514	PASS
Phase 2	114514	PASS
Phase 3	114514	PASS
Phase 4	114514	PASS
Phase 5	114514	PASS
Phase 6	114514	PASS

表 4: 非公开 seed 验证记录

8.3 运行结果截图

```
-- sky

窗口角落留着‘返回实验室’的按钮。
林夏把外套搭在椅背上：‘先看看？’
维护面板上画着一枚炸弹，旁边亮着六项未完成的交接检查。
它不是街上的爆炸物，只是旧系统给这次维护选的图标。

第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0xc3e450079c288d47
Input (unsigned decimal integers):
2186505535
Phase 1 defused.

门锁响了一声，维护站开始接收新的记录。
日志里的请求时间各不相同，城市的日期却始终是同一天。
其中一份校时记录来自街那头的钟楼。林夏记下地址，推开门。

[1 已解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
```

图 3: Phase 1：非公开 seed 运行结果

```
第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0xed3459d3bab69fb5
Input (unsigned decimal integers):
237 1189 5956 29811 149060 745315
Phase 2 defused.

校时检查通过了。钟楼送出一条新记录，几秒后又收回原样的一条。
‘不是钟没走。’林夏把两条记录放在一起，‘它送出去又回来了。’
你们沿着线路往下查，重复的地址最后指向街口的分流站。

[1 已解除] [2 已解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
```

图 4: Phase 2：非公开 seed 运行结果

```
[1 已解除] [2 已解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0x6e51758b9acd24fd
Input (unsigned decimal integers):
5 974204681
Phase 3 defused.

这次请求没有再绕回来。分流站把它送进档案馆，收到一条回复：
‘未找到交接存档。’回复后面附着一份旧目录。
林夏收好路线图。你们穿过马路，档案馆的门刚刚打开。

[1 已解除] [2 已解除] [3 已解除] [4 当前] [5 待解除] [6 待解除]

第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
```

图 5: Phase 3：非公开 seed 运行结果

```
第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

Phase 4 challenge: 0x077b3d42a7534dc6
Input (unsigned decimal integers):
703942541
Phase 4 defused.

最后一层索引返回了一份完整存档，还有那次迁移留下的记录。
新设备接上时，交接没有做完。城市只能反复恢复最后一份旧存档；
居民今天做过的事，到了夜里就都没了。
存档后面列着几处还没核对的设备。你认出了维修铺那盏灯。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 当前] [6 待解除]

第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口
```

图 6: Phase 4：非公开 seed 运行结果

```
第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0xfcac85f876912d1c
Input (unsigned decimal integers):
1385719713 296
Phase 5 defused.

最后一处连接也对上了，今天的设备状态第一次进入待保存的记录。
维修铺老板试着拧了一下灯座，灯亮了一瞬，又被他小心地关掉。
‘留到明天装好。’他说。
交接清单上只剩最后一项。你们带着记录走到回廊尽头。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 当前]

第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0x2055abe6336eebb1
```

图 7: Phase 5：非公开 seed 运行结果



图 8: Phase 6：非公开 seed 运行结果



图 9: 非公开 seed：114514

8.4 Secret 验证

Secret 已通过十个公开 seed 的测试。

9 辅助工具与参考资料

9.1 辅助工具

实验中使用的主要工具如下：

工具	用途
GDB	查看函数反汇编、寄存器、跳转表、函数指针表和链表节点。
objdump	导出目标程序的汇编代码，辅助观察整体指令序列。
VS Code	编写 C 程序、整理汇编文本及查看测试结果。
make	编译实验程序。
public_test.py	验证不同 seed 下的计算结果。
ChatGPT	辅助理解汇编指令、讨论计算公式和程序结构、排查数据抄写错误及整理报告草稿。

表 5: 实验辅助工具及用途

9.2 参考资料

1. 复旦大学计算机系统基础（2026 秋），Lab2: BombLab 实验说明：
   https://ics-26fall-fdu.github.io/labs/lab2-bomb-lab/
2. 实验仓库 README.md、teaching 示例及相关源代码。

10 实验总结

本实验通过分析 x86-64 汇编代码，恢复了六个常规关卡和隐藏关的计算关系。

分析过程中，比较重要的经验是：

1. `lea` 不一定是在计算地址，也可能是编译器实现整数运算的方式。
2. `shr`、`ror`、`rol` 等指令需要严格区分，同时注意 32 位运算的回绕行为。
3. 对于跳转表和函数指针表，仅阅读汇编代码还不够，需要检查实际内存中的地址对应关系。
4. 恢复链表时，除了理解遍历算法，还必须准确确认字段偏移、指针关系和节点的静态数据。
5. 递归函数需要区分递归终止条件、递归调用和返回值处理。

最终实现的算法能够处理不同的 `challenge`，六个常规关卡的公开测试全部通过，隐藏关的公开测试也全部通过。